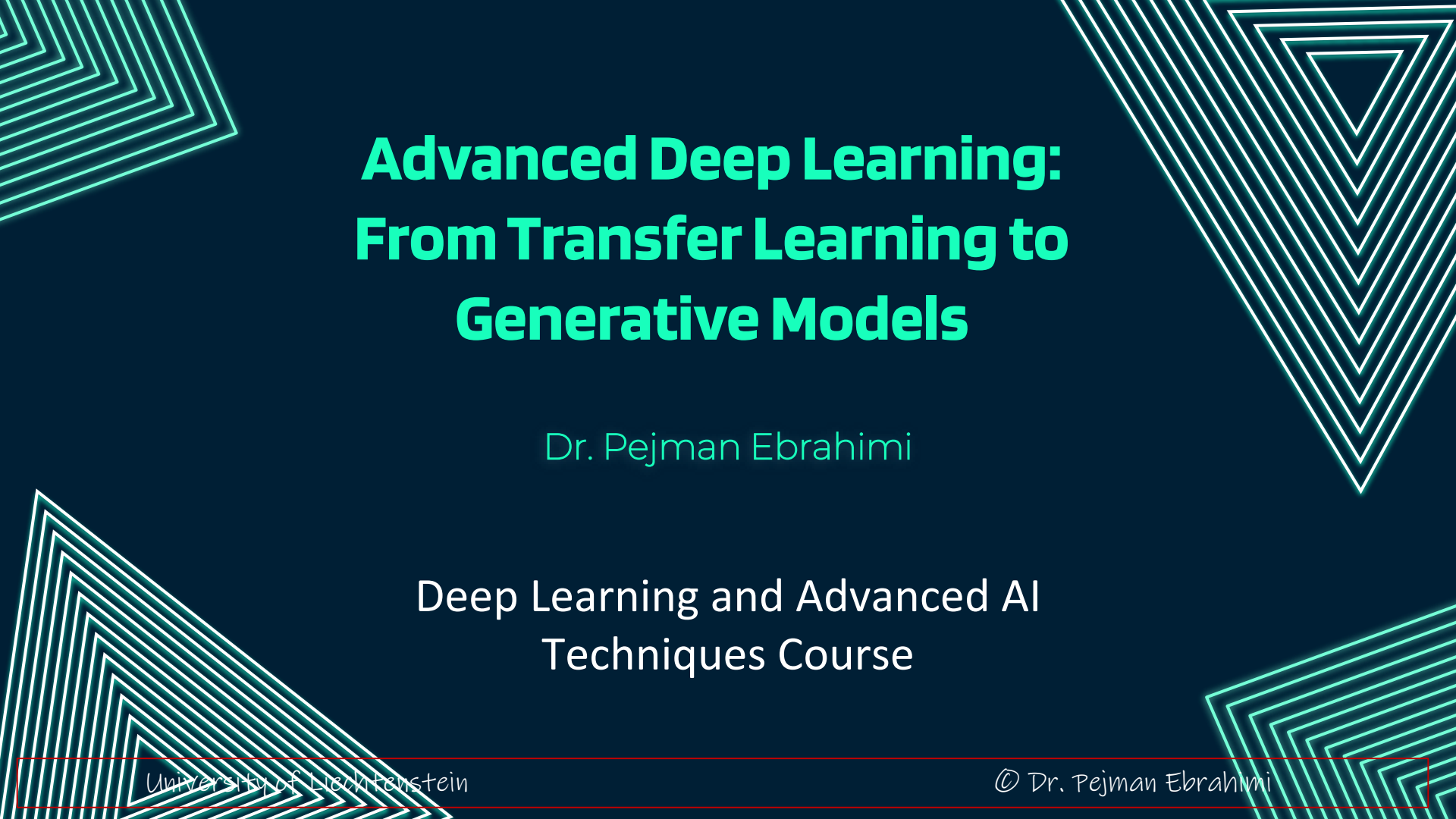


The slide features a dark blue background with four decorative corner elements. Each element consists of multiple concentric, nested triangles or chevrons pointing towards the center. The lines are thin and light blue, creating a subtle geometric pattern in each corner.

Advanced Deep Learning: From Transfer Learning to Generative Models

Dr. Pejman Ebrahimi

Deep Learning and Advanced AI
Techniques Course

A series of concentric hexagons, each outlined in a light blue color, creating a tunnel-like effect that draws the eye towards the center. The hexagons are slightly offset from each other, giving a 3D perspective.

01

Convolutional Neural Networks (CNNs)

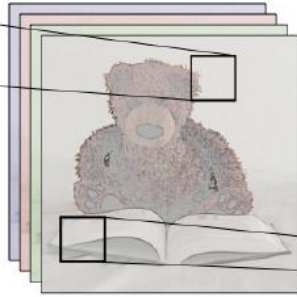
Convolutional Neural Networks (CNNs)

- ❑ **Definition:** Neural networks with convolutional layers designed to process grid-like data
- ❑ **Key Innovation:** Parameter sharing and local connectivity
- ❑ **Core Components:**
 - ✓ Convolutional layers: Extract features using filters/kernels
 - ✓ Pooling layers: Downsample spatial dimensions
 - ✓ Fully connected layers: Classification based on extracted features
- ❑ **Main Applications:** Image processing, computer vision tasks

Architecture of a traditional CNN



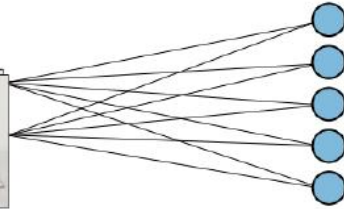
Input image



Convolutions



Pooling



Fully Connected

Input image

Convolutions

Pooling

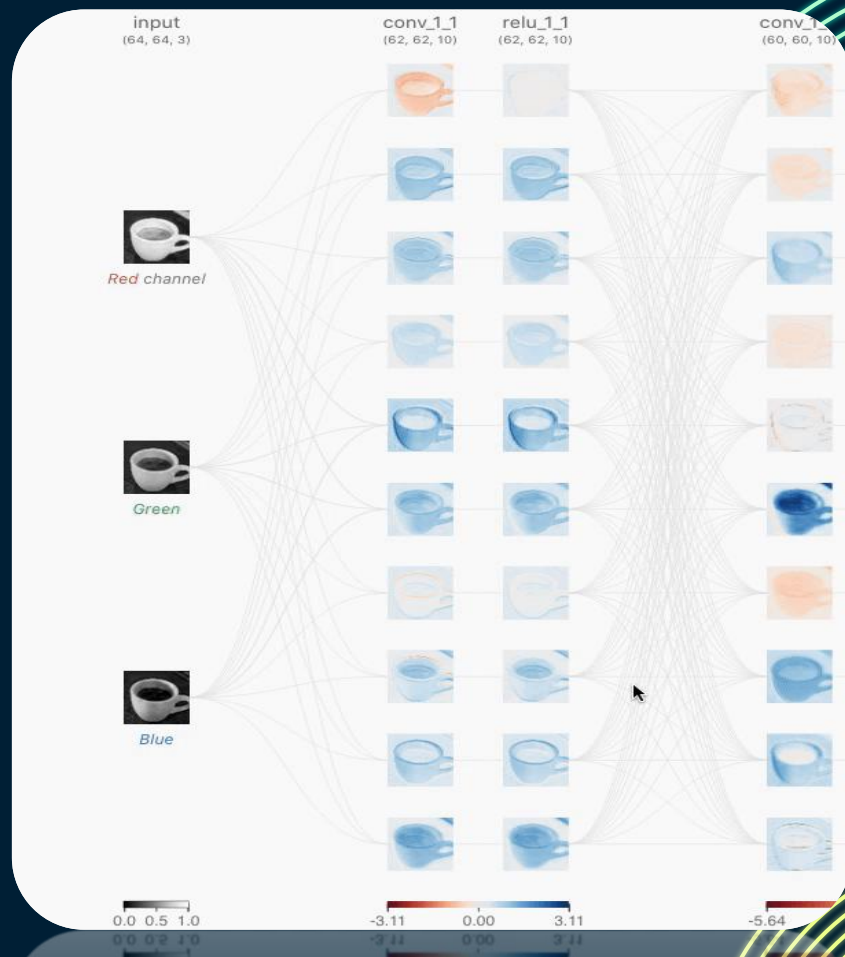
Fully Connected

Structure of a CNN

- ❑ **Input Layer** – the image (e.g., 3 color channels: **RGB**)
- ❑ **Convolutional Layers** – extract features
- ❑ **Pooling Layers** – downsample the features
- ❑ **Flatten Layer** – prepares data for classification
- ❑ **Fully Connected Layers** – make the final decision
- ❑ **Output Layer** – gives class probabilities using **Softmax** or **Sigmoid**

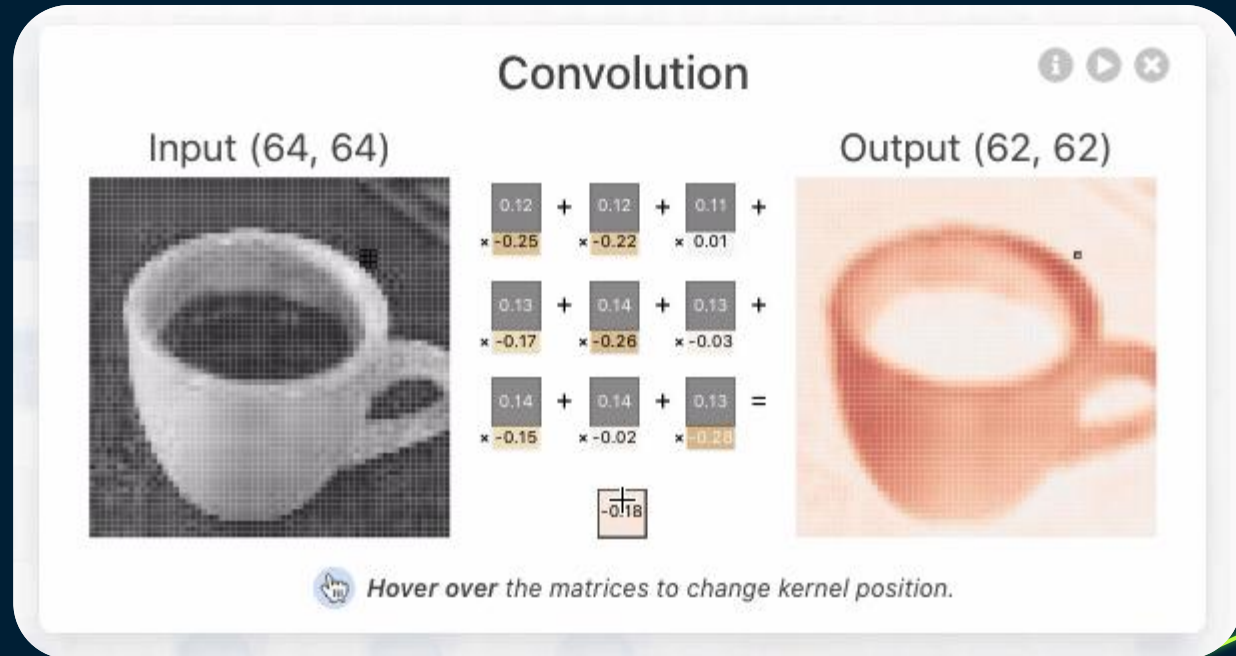
Convolutional layer (CONV)

- ❑ Detects **patterns** like edges, textures, shapes.
- ❑ Uses small **filters (kernels)** to slide over the image.
- ❑ Performs a **dot product** with image pixels.
- ❑ Outputs an **activation map (feature map)**.



CNN Hyperparameters

- ❑ **Kernel Size:** Smaller = more local features
- ❑ **Stride:** Larger = faster, smaller output
- ❑ **Padding:** Preserves image size





Input Size:

5



Padding:

0



Kernel Size:

4

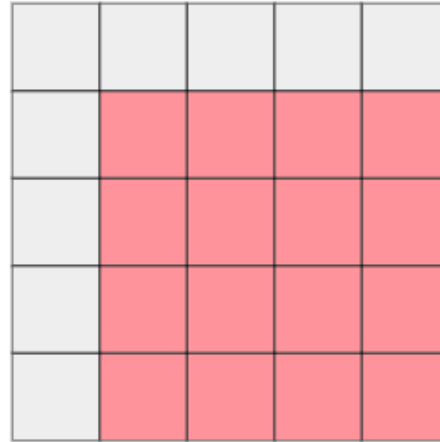


Stride:

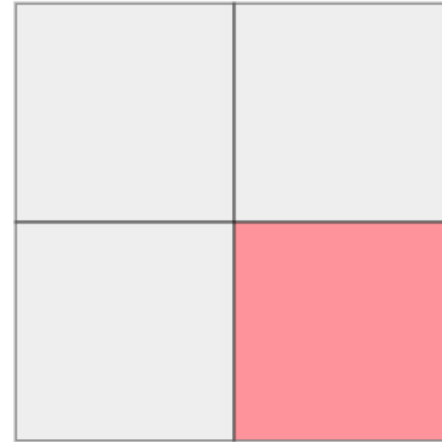
1



Input (5, 5)
After-padding (5, 5)



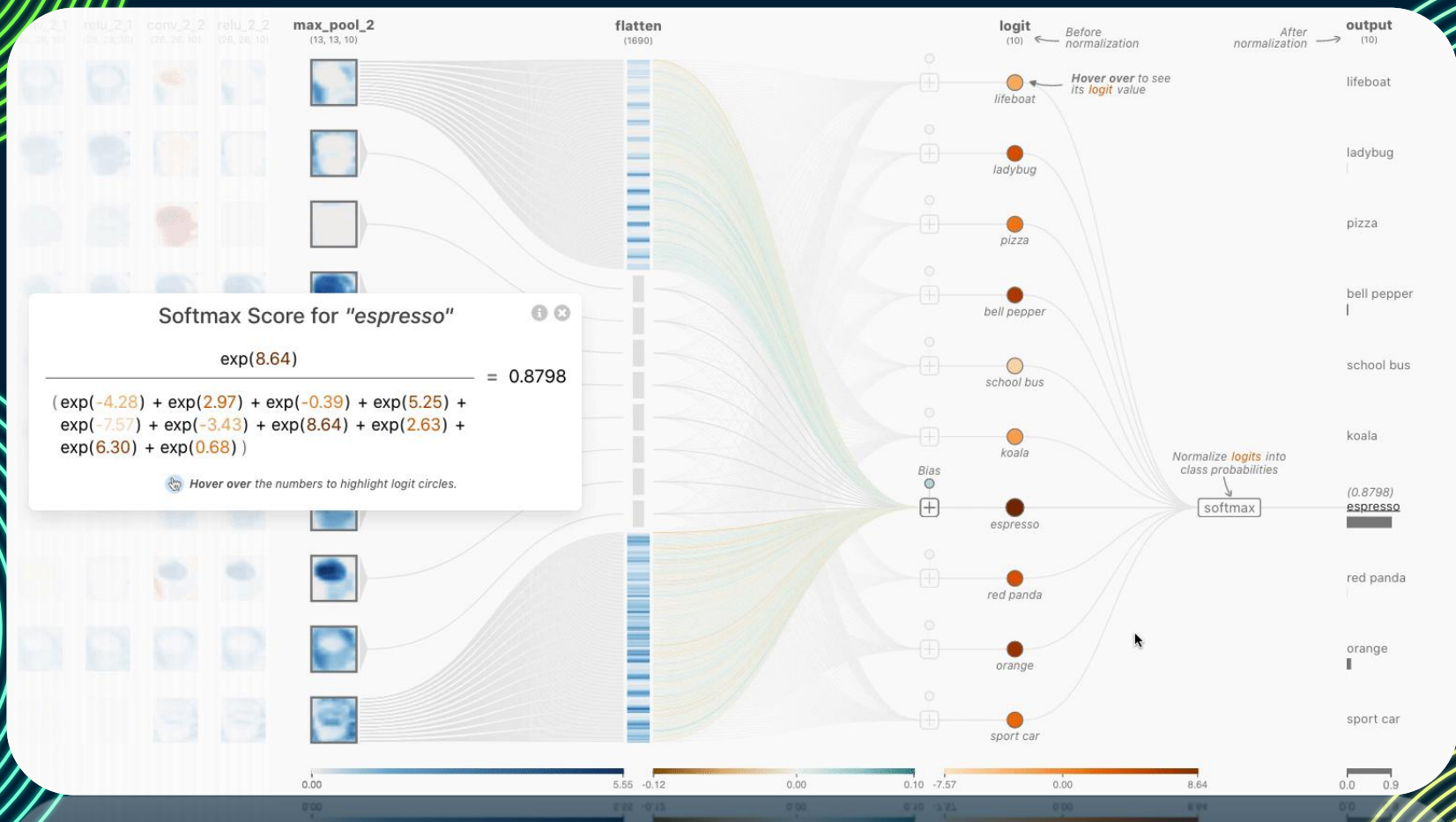
Output (2, 2)



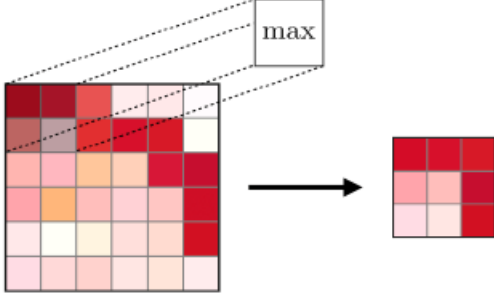
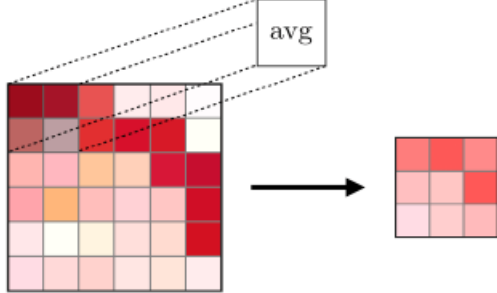
Hover over the matrices to change kernel position.

Pooling layer (POOL)

- ❑ Simplifies the feature maps.
- ❑ **Max pooling** picks the biggest number in each patch.
- ❑ Reduces **size**, computation, and overfitting.
- ❑  Example: 2x2 max-pooling on [3, 7, 1, 5] → keeps 7



Max pooling & Average pooling

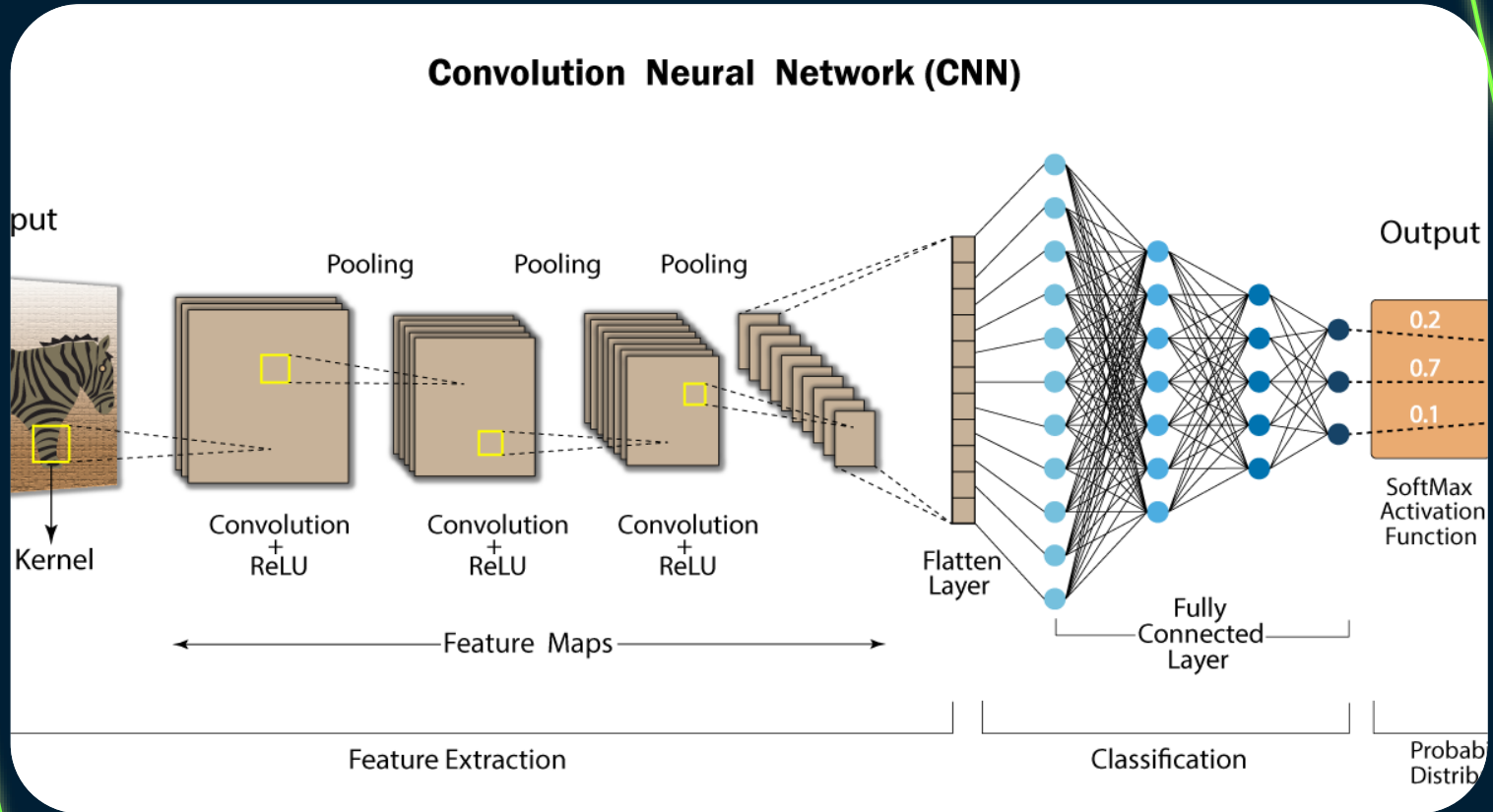
	Max pooling	Average pooling
Purpose	Each pooling operation selects the maximum value of the current view	Each pooling operation averages the values of the current view
Illustration		
Comments	- Preserves detected features - Most commonly used	- Downsamples feature map - Used in LeNet

Flatten and Fully Connected Layer

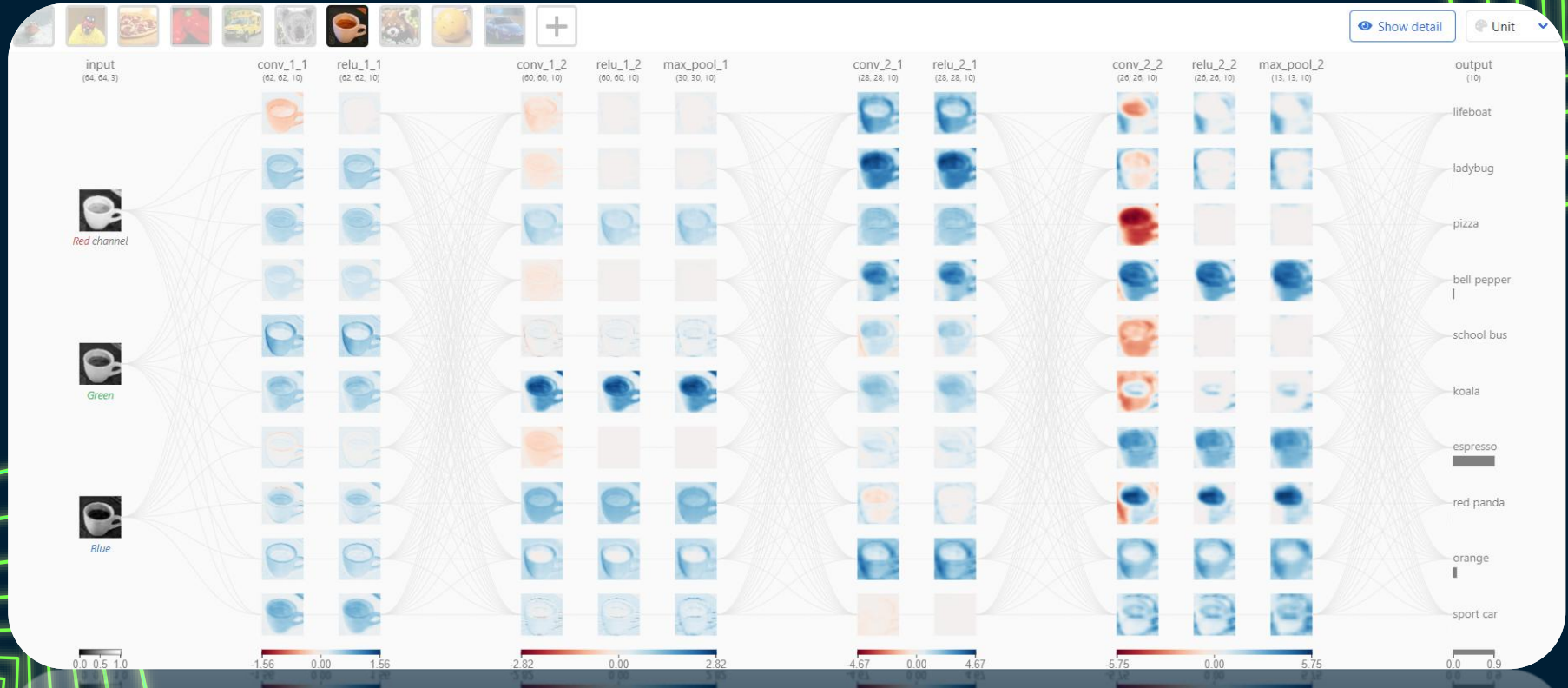


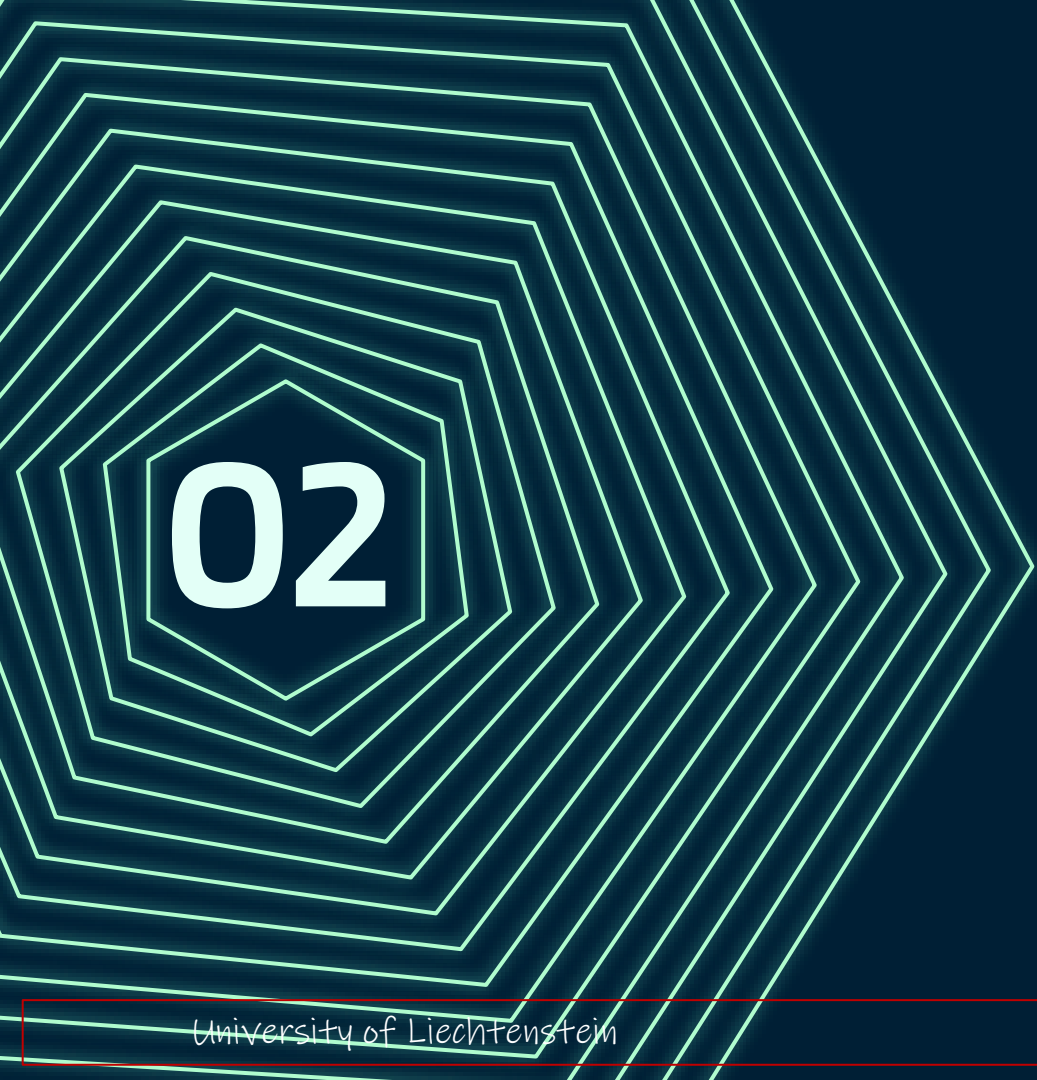
- ❑ Flatten Layer turns 3D data into 1D vector.
- ❑ Fully Connected Layer processes this vector.
- ❑ Final layer uses Softmax (or sigmoid) to assign class probabilities.
- ❑ 🧠 Example: Flatten 5x5x2 → 1D vector of size 50

Recap – How CNN Works



CNN Explainer Playground

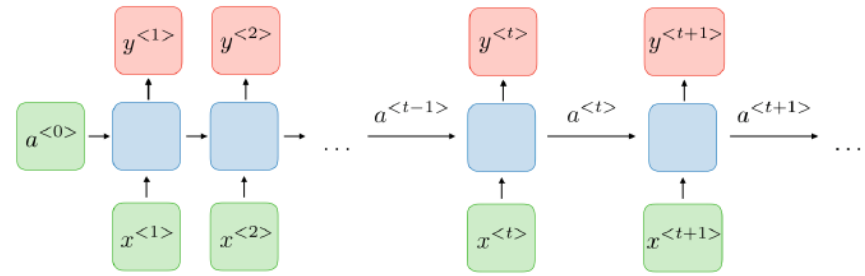


A decorative graphic on the left side of the slide consisting of multiple concentric hexagons. The innermost hexagon is dark blue and contains the white text '02'. The subsequent hexagons are outlined in a light blue color and expand outwards, creating a tunnel-like effect.

02

Recurrent Neural Networks (RNNs)

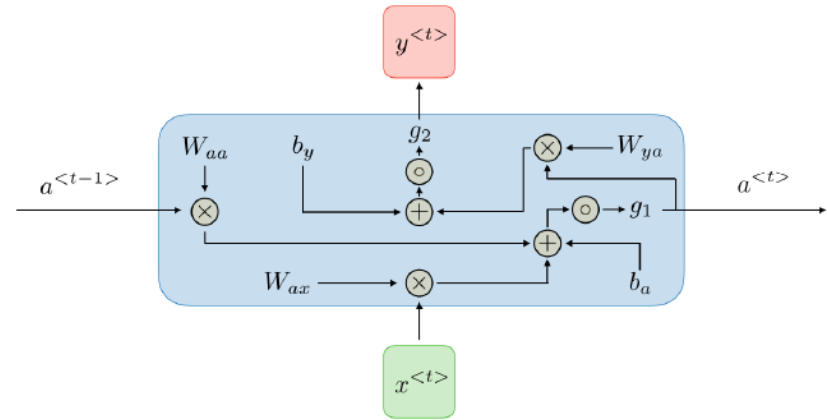
Architecture of a traditional RNN



For each timestep t , the activation $a^{<t>}$ and the output $y^{<t>}$ are expressed as follows:

$$a^{<t>} = g_1(W_{aa}a^{<t-1>} + W_{ax}x^{<t>} + b_a) \quad \text{and} \quad y^{<t>} = g_2(W_{ya}a^{<t>} + b_y)$$

where $W_{ax}, W_{aa}, W_{ya}, b_a, b_y$ are coefficients that are shared temporally and g_1, g_2 activation functions



What is a Recurrent Neural Network (RNN)?

- ❑ RNNs are designed to handle sequential data like text, time series, or audio (**natural language processing** and **speech recognition**).
- ❑ Unlike CNNs, RNNs remember information using loops in their architecture.
- ❑ Great for tasks where order matters, like predicting the next word in a sentence.
- ❑ 🧠 Example: “**I love ___**” → predicts “**you**” or “**pizza**” based on previous words.

Types of RNN

Type of RNN	Illustration	Example
One-to-one $T_x = T_y = 1$		Traditional neural network
One-to-many $T_x = 1, T_y > 1$		Music generation
Many-to-one $T_x > 1, T_y = 1$		Sentiment classification
Many-to-many $T_x = T_y$		Name entity recognition
Many-to-many $T_x \neq T_y$		Machine translation

LSTM and GRU

- Traditional RNNs forget long-term info — LSTM and GRU solve this

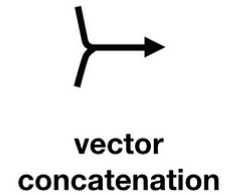
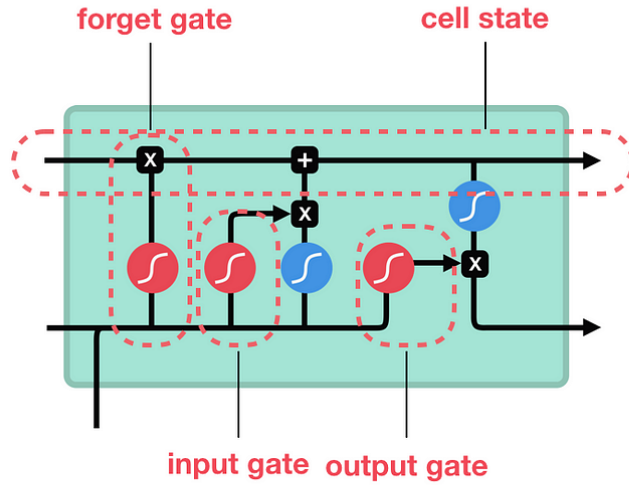
LSTM (Long Short-Term Memory):

- Has memory cells and gates (forget, input, output)
- Remembers information for long durations

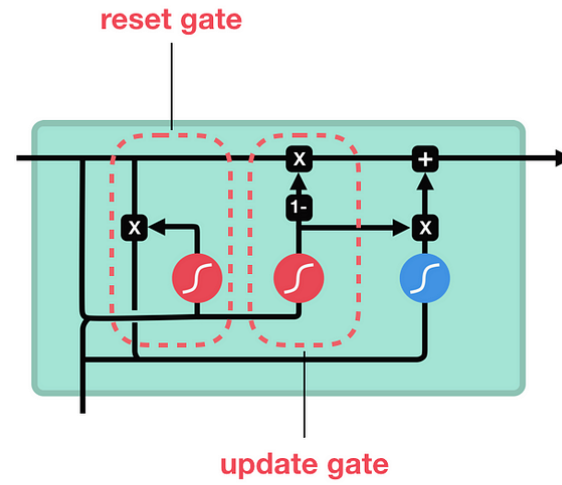
GRU (Gated Recurrent Unit):

- A simpler version of LSTM with fewer gates
- Often faster and similarly effective

LSTM



GRU



Bidirectional RNNs

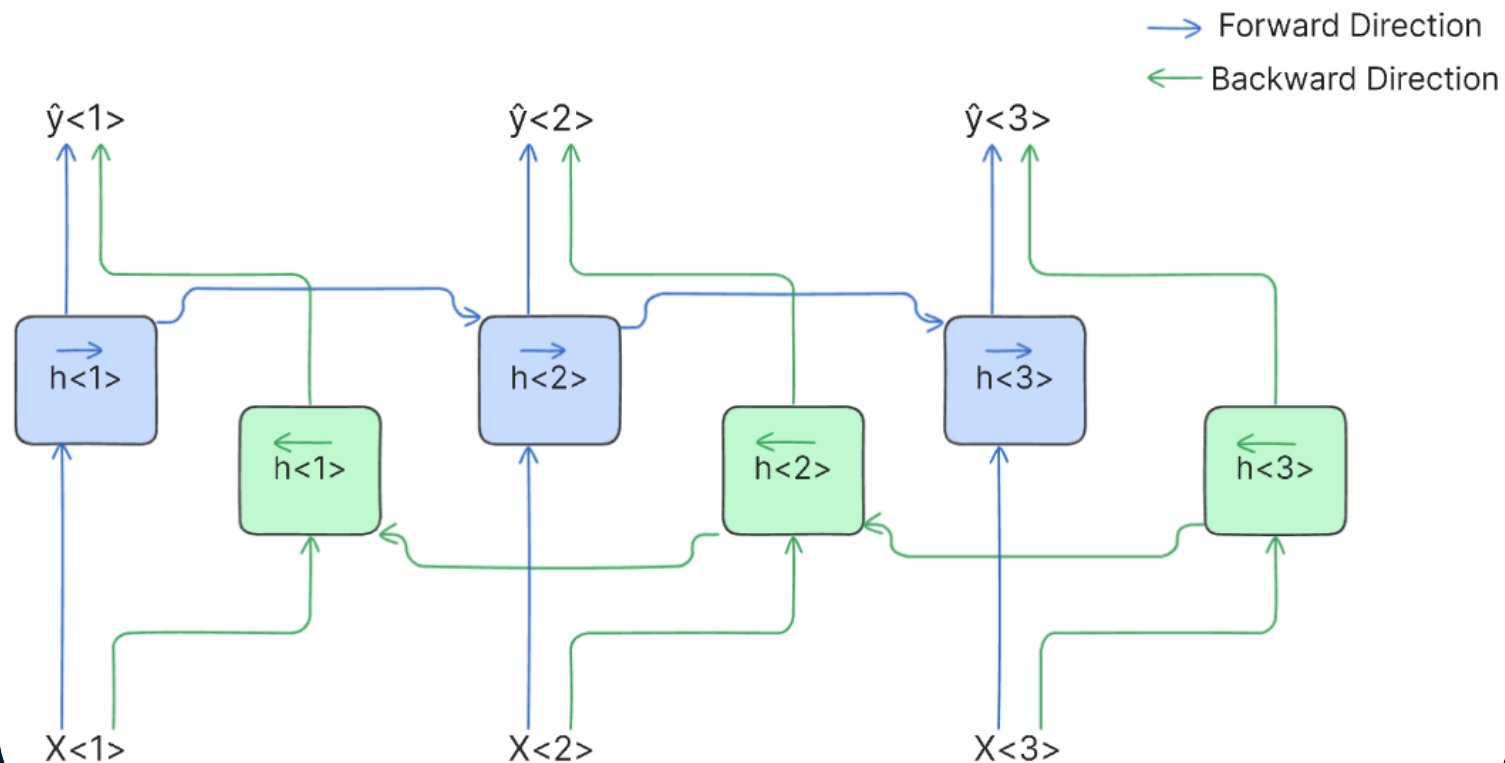
❑ What are Bidirectional RNNs?

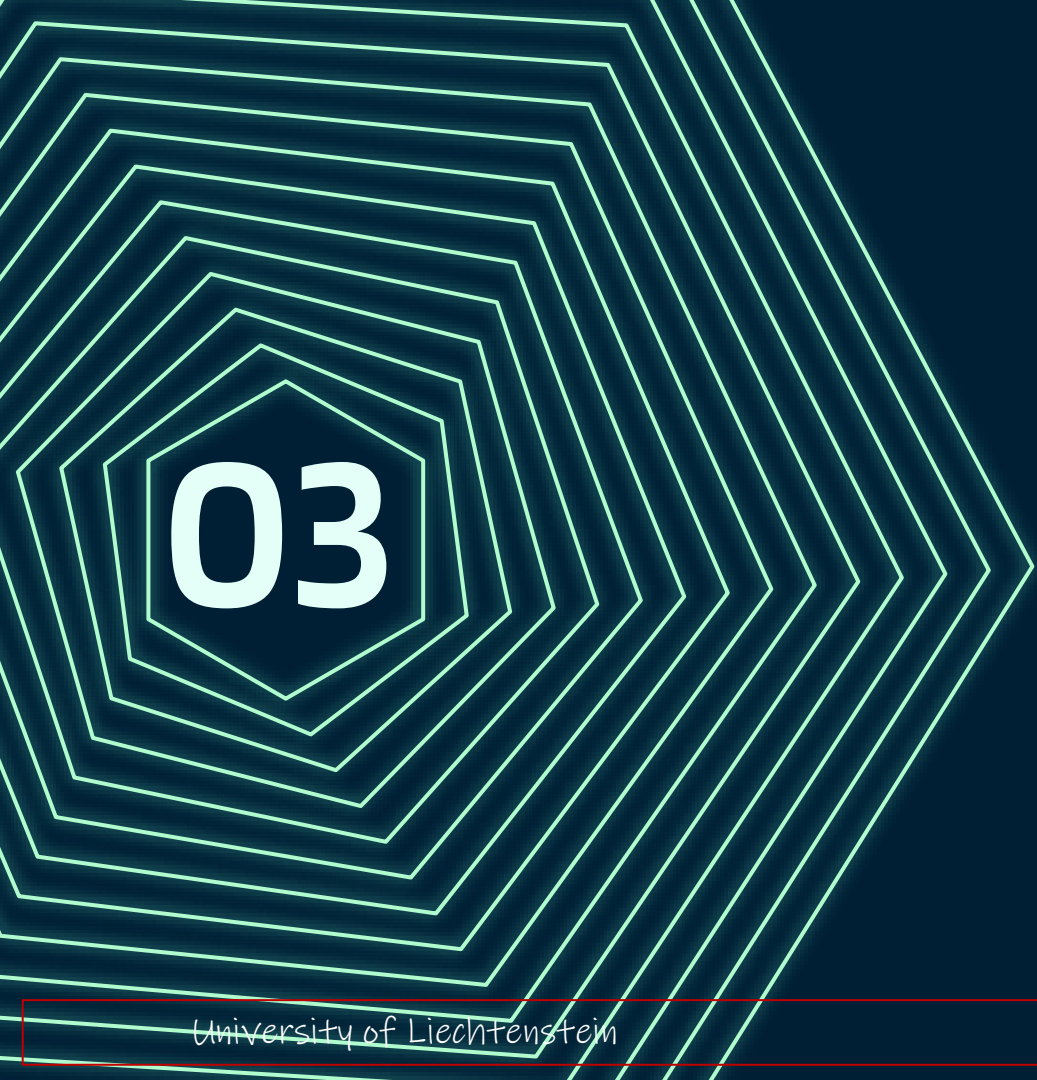
- Standard RNNs process data **left to right** (past → future).
- **Bidirectional RNNs** process data in **both directions**:
 - ✓ One RNN moves **forward** (→)
 - ✓ Another moves **backward** (←)
- Their outputs are **combined** (e.g., concatenated) for each time step.

❑ 🎯 Why useful?

- ✓ They understand context from **both past and future**.
- ✓ Improves accuracy in tasks like **speech recognition** and **named entity recognition**.

Bidirectional RNN Architecture



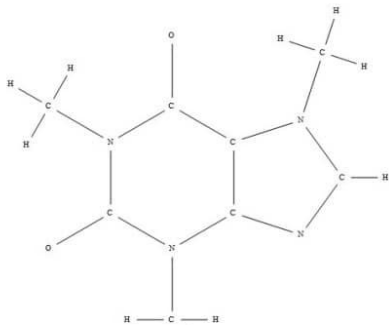
A series of concentric hexagons, each outlined in a light blue color, creating a tunnel-like effect that draws the eye towards the center. The hexagons are slightly offset from each other, giving a 3D or layered appearance. The number '03' is centered within the innermost hexagon.

03

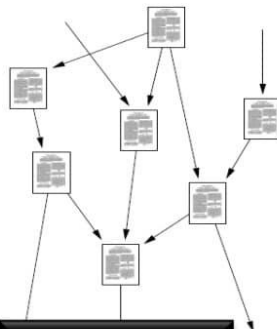
Graph Neural Networks (GNNs)

Graph Neural Networks (GNNs)

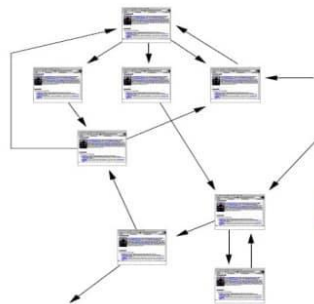
- ❑ **Definition:** Neural networks that operate on graph-structured data
- ❑ **Key Innovation:** Message passing between nodes based on graph connectivity
- ❑ **Core Components:**
 - Node features: Attributes of entities
 - Edge features: Relationship attributes
 - Message passing: Information exchange along edges
- ❑ **Applications:** Social networks, molecular structures, knowledge graphs, recommender systems



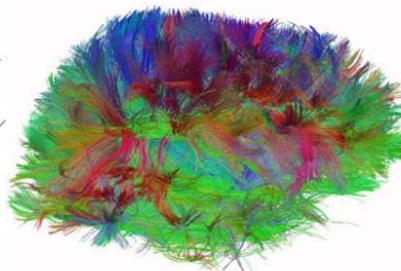
Molecules



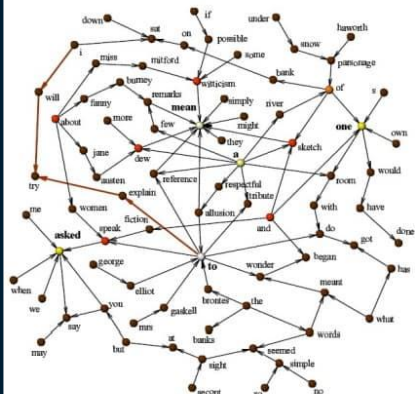
Knowledge



Information



Brain/neurons



Genes

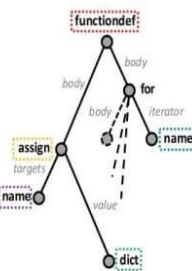


Communication

```
def encode(obj):
    """
    Encode a (possibly nested)
    dictionary containing complex values
    into a form that can be serialized
    using JSON.
    """
    e = {}
    for key, value in obj.items():
        if isinstance(value, dict):
            e[key] = encode(value)
        elif isinstance(value, complex):
            e[key] = {'type': 'complex',
                    'r': value.real,
                    'i': value.imag}
    return e

import ast
tree = ast.parse("""
```

Software



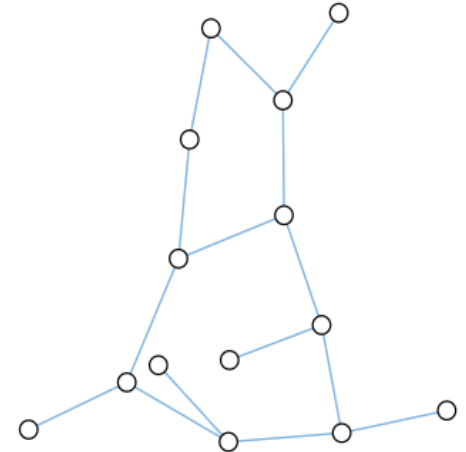
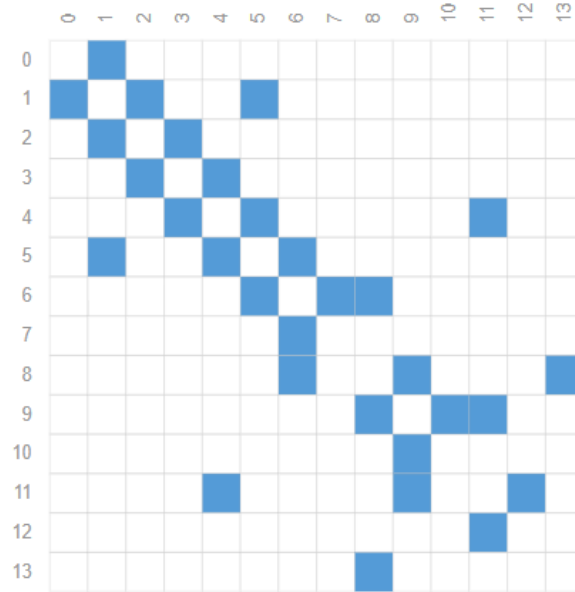
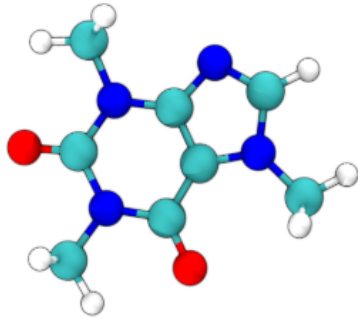
Social

We can digitize text by associating indices to each character, word, or token, and representing text as a sequence of these indices. This creates a simple directed graph, where each character or index is a node and is connected via an edge to the node that follows it.



	Graphs	are	all	around	us
Graphs		1			
are			1		
all				1	
around					1
us					

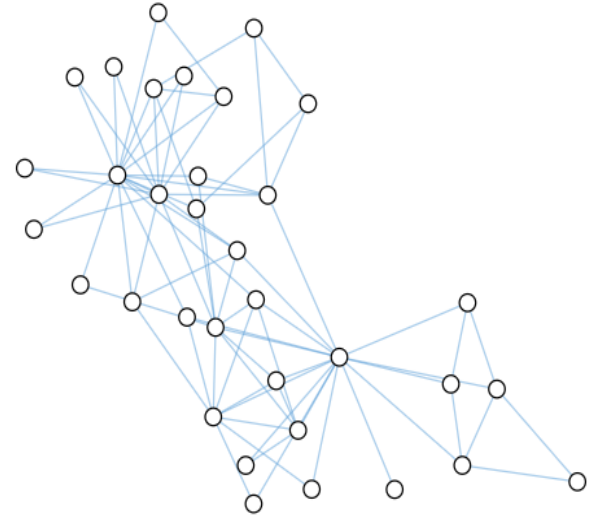
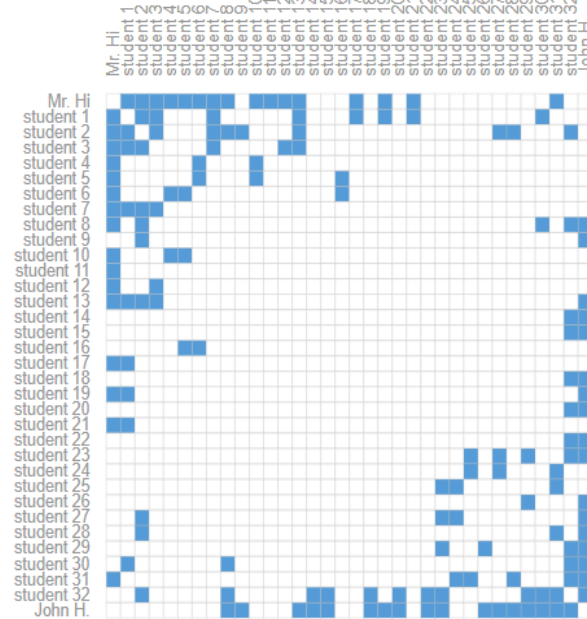
Molecules as graphs



(Left) 3d representation of the Caffeine molecule (Center) Adjacency matrix of the bonds in the molecule (Right) Graph representation of the molecule.

(left) 3d representation of the Caffeine molecule (Center) Adjacency matrix of the bonds in the molecule (right) Graph representation of the molecule

Image as graphs



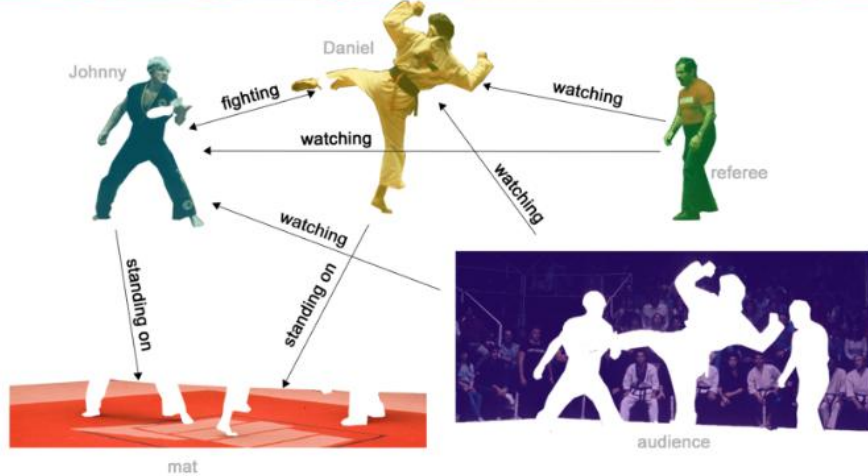
(Left) Image of karate tournament. (Center) Adjacency matrix of the interaction between people in a karate club. (Right) Graph representation of these interactions.

(left) image of karate tournament (center) adjacency matrix of the interaction between people in a karate club (right) graph representation of these interactions

Edge-level task

One example of edge-level inference is in image scene understanding. Beyond identifying objects in an image, deep learning models can be used to predict the relationship between them.

We can phrase this as an edge-level classification: given nodes that represent the objects in the image, we wish to predict which of these nodes share an edge or what the value of that edge is.

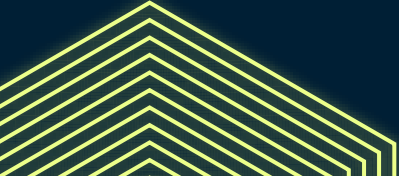


In (b), above, the original image (a) has been segmented into five entities: each of the fighters, the referee, the audience and the mat. (C) shows the relationships between these entities.



⚡ Quiz – CNN, RNN, and GNN Applications

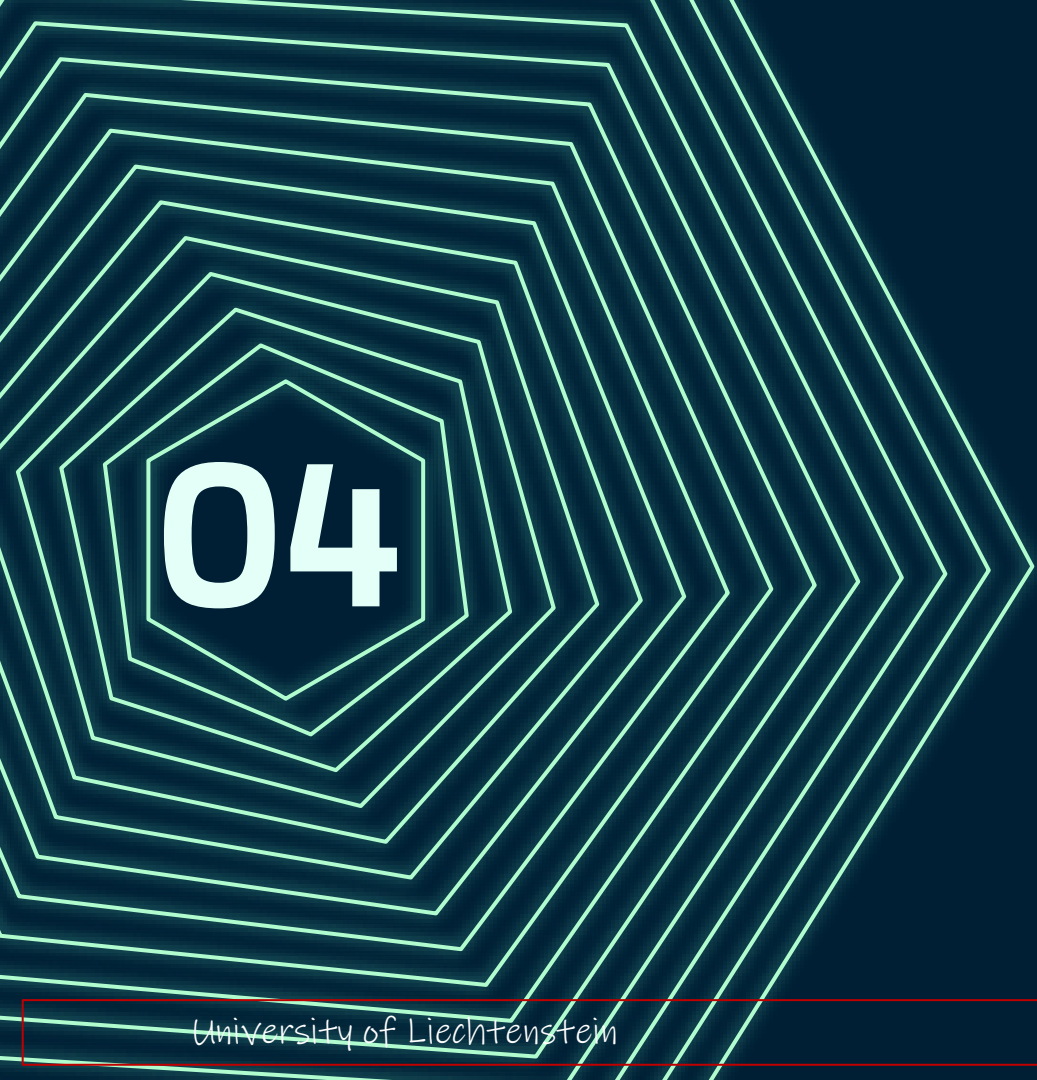
Match the architecture to its most suitable application:

- ☐ Analyzing CT scan images for tumor detection
 - ☐ Predicting next word in a sentence
 - ☐ Drug discovery by analyzing molecular structures
 - ☐ Speech recognition
 - ☐ Recommender systems based on user interactions
- 

Task

🕒 Prepare a 10-minute presentation

- ❑ Explaining the main **types of Graph Neural Networks (GNNs)**: such as GCN, GraphSAGE, GAT, and GIN
- ❑ Discuss **key challenges** in GNNs, including over-smoothing, scalability, and interpretability, with relevant examples.

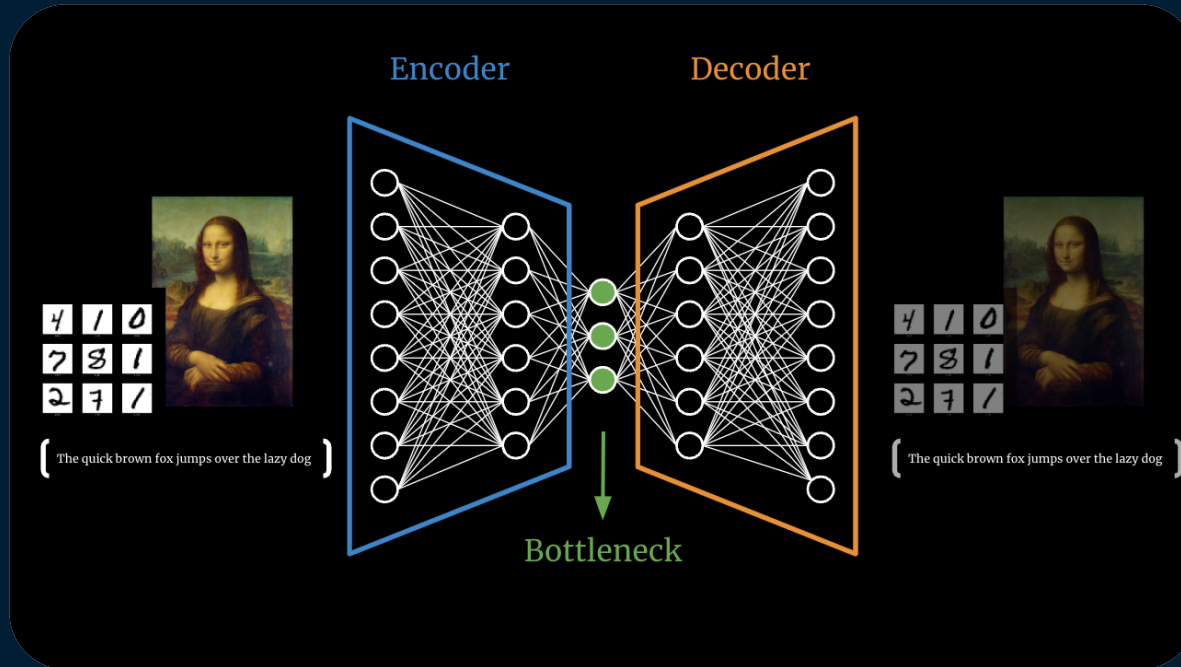
A series of concentric hexagons drawn with thin, light blue lines, creating a tunnel-like effect that draws the eye towards the center. The hexagons are slightly offset from each other, giving a 3D or layered appearance.

04

Autoencoders

What is an Autoencoder?

- ❑ **An autoencoder** is a type of neural network that learns how to compress data and then rebuild it.



1. Encoder:

- Takes the input (like an image) and **compresses** it into a smaller version called a **latent space** (or bottleneck).
- It keeps only the important information.

2. Latent Space:

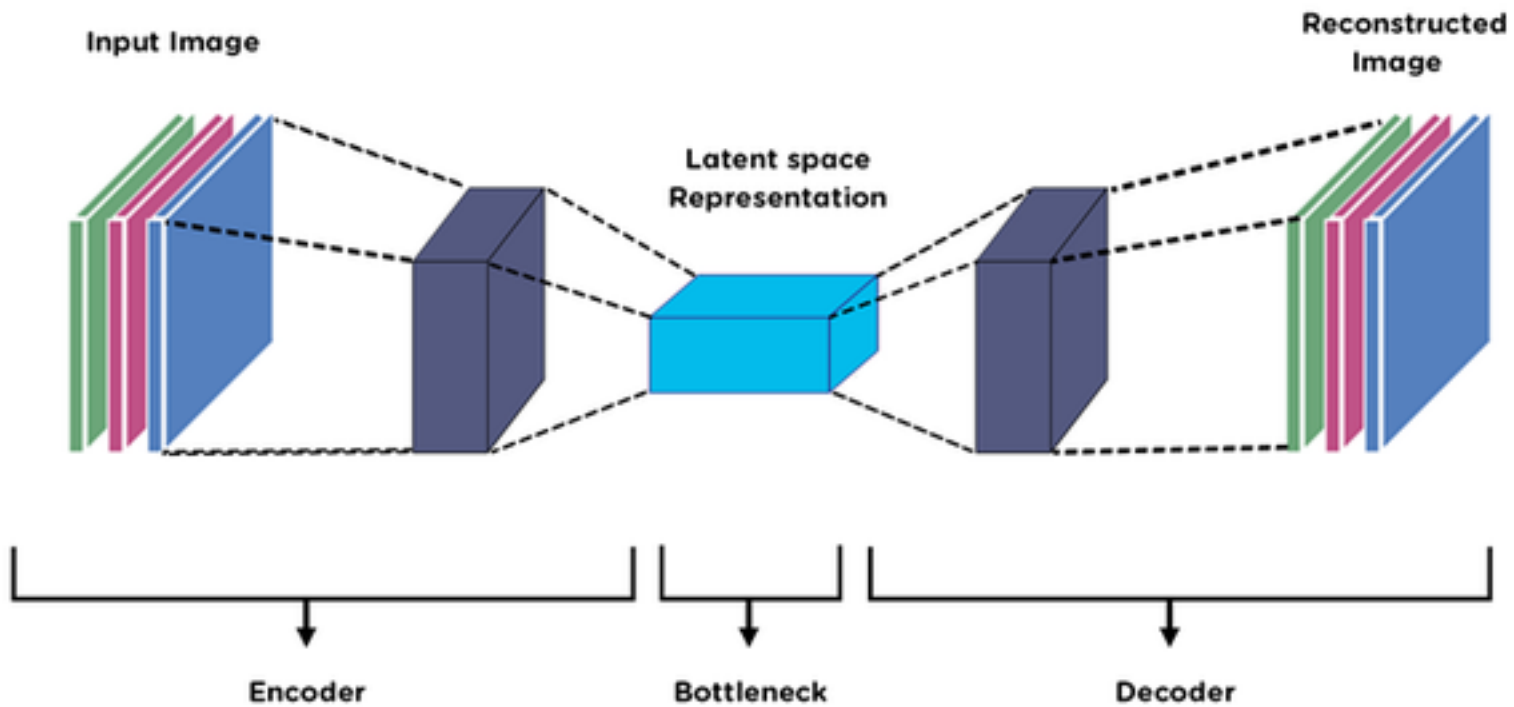
- A short, compact representation of the input.
- Like the “essence” of the input.

3. Decoder:

- Tries to **rebuild the original input** from the compressed version.
- The goal is to make the output look as close to the input as possible.

• The goal is to make the output look as close to the input as possible.

• Tries to rebuild the original input from the compressed version.



Why use Autoencoders?

- ❑ **Noise removal (denoising images).**
- ❑ **Dimensionality reduction (like PCA but smarter).**
- ❑ **Anomaly detection (bad reconstructions = weird data).**
- ❑ **A stepping stone to generative models like GANs.**

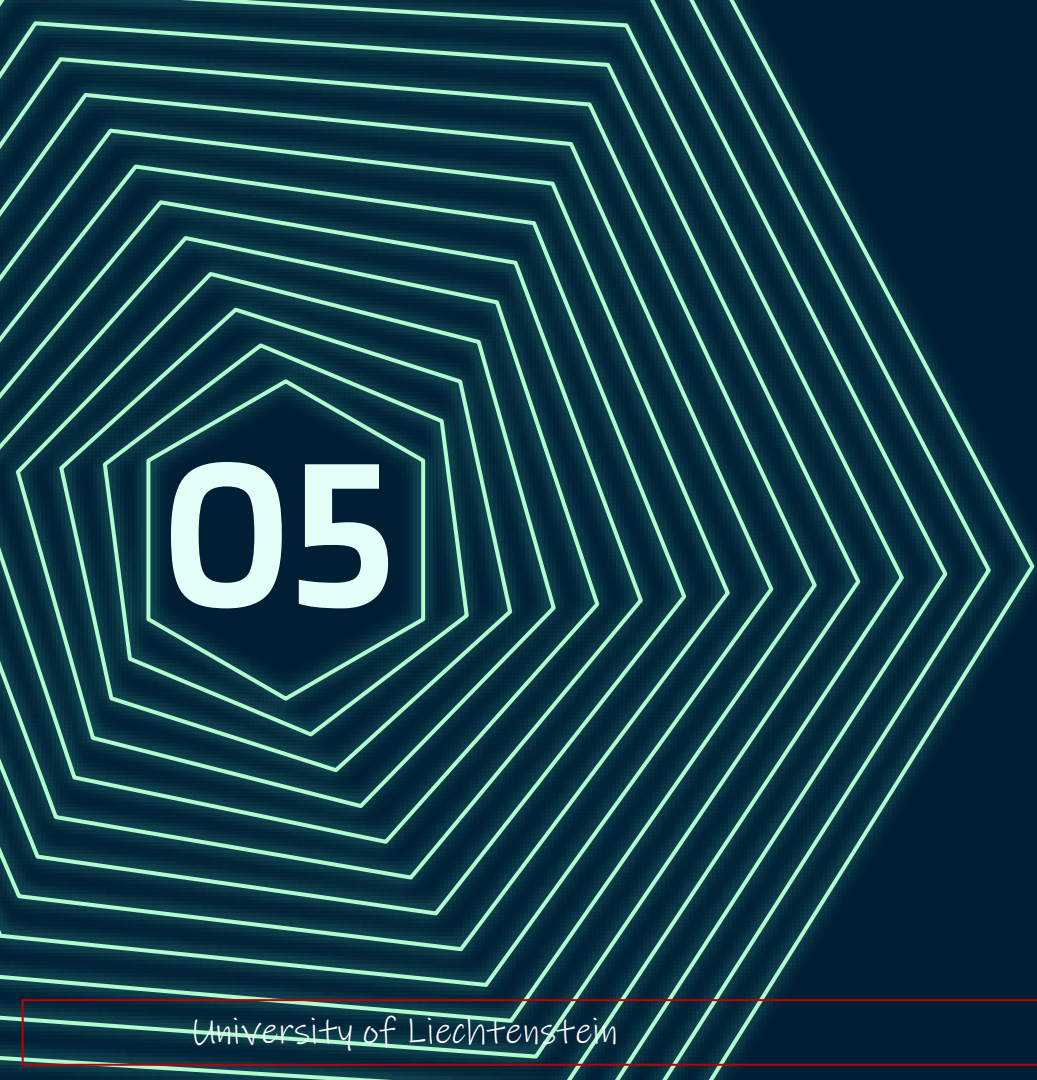
Denoising Autoencoders

- ❑ **The aim** of Denoising Autoencoders is to remove noise from an image.



Autoencoder Variants

- ❑ **Vanilla Autoencoder**: Basic compression and reconstruction
- ❑ **Denoising Autoencoder**: Add noise to input, learn to reconstruct clean version Builds robustness to noise and corruption
- ❑ **Sparse Autoencoder**: Adds sparsity constraint on activations Forces learning of more efficient representations
- ❑ **Variational Autoencoder (VAE)**: Probabilistic approach to encoding Learns distribution in latent space Enables generative capabilities
- ❑ **Contractive Autoencoder**: Penalizes sensitivity to input variations Creates more robust representations

A series of concentric hexagons drawn with thin, light blue lines, creating a tunnel-like effect that draws the eye towards the center. The hexagons are slightly offset from each other, giving a 3D or layered appearance.

05

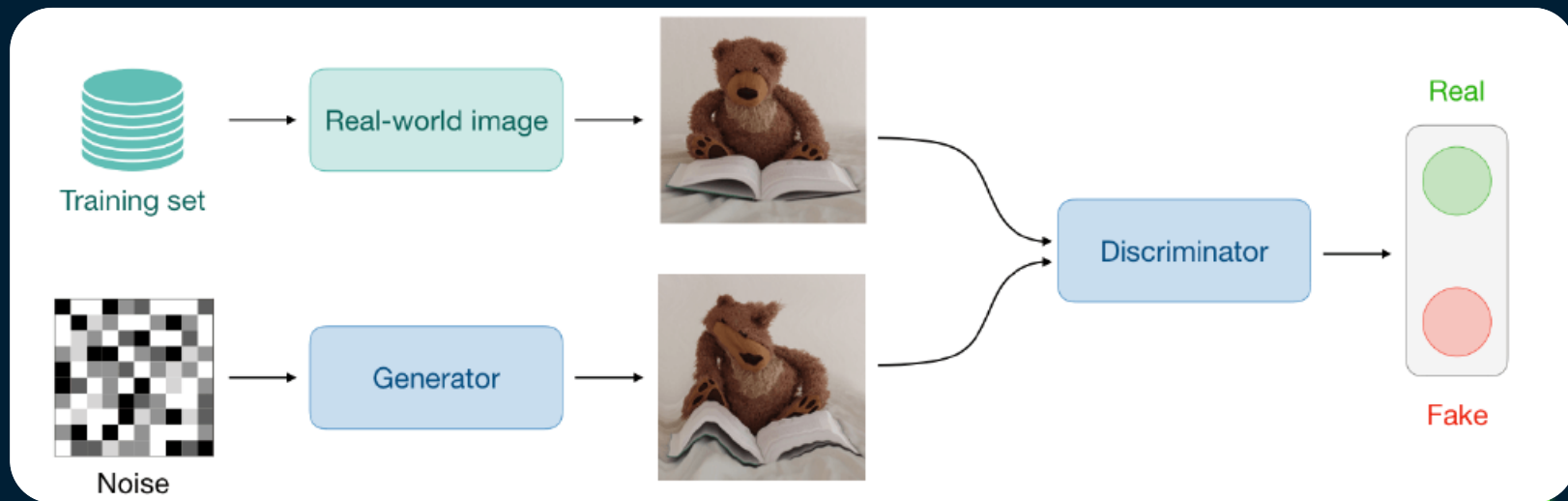
Generative Adversarial Networks (GANs)

GANs

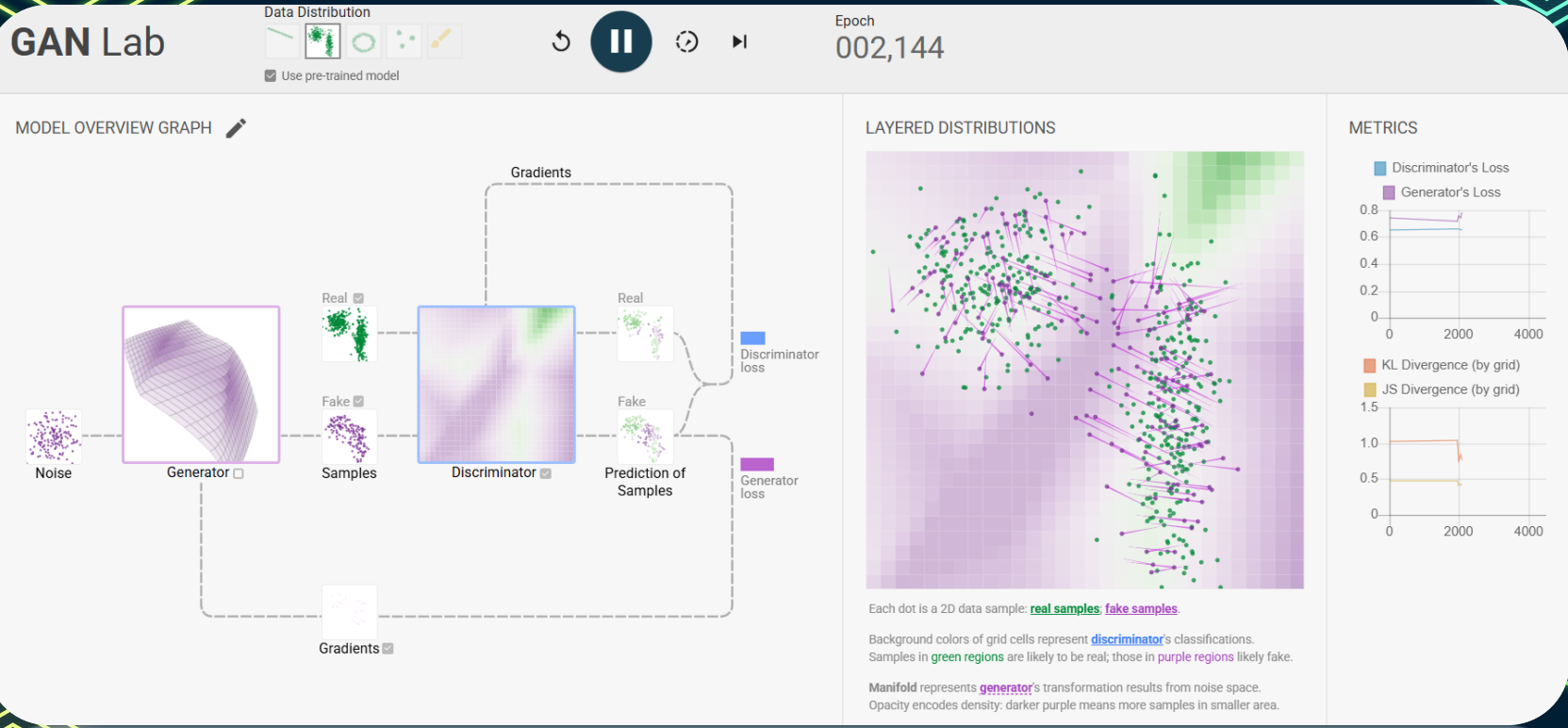
- ❑ **Definition:** Framework where two networks compete: a generator creates data and a discriminator evaluates authenticity
- ❑ **Key Innovation:** Adversarial training paradigm
- ❑ **Components:**
 - **Generator:** Creates synthetic samples from random noise
 - **Discriminator:** Classifies samples as real or fake
- ❑ **Training Process:**
 - **Zero-sum game (minimax)**
 - **Generator tries to fool**
 - **discriminator Discriminator tries to correctly identify fakes**

Applications of GANs

- ❑ **Image Synthesis:** Generate realistic faces, art, scenes.
- ❑ **Data Augmentation:** Generate samples for imbalanced datasets.
- ❑ **Style Transfer:** Apply artistic styles to images.
- ❑ **Super-Resolution:** Enhance image resolution.
- ❑ **Medical Imaging, Text-to-Image, 3D Object Generation, etc.**



GAN Playground

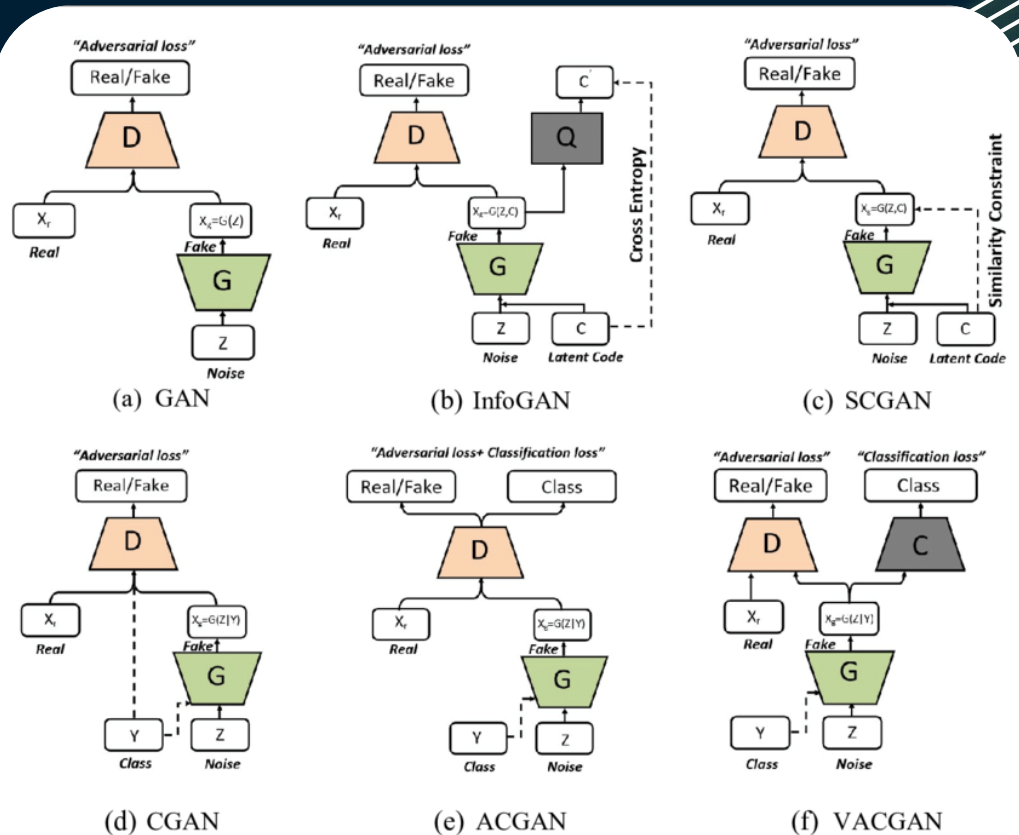


GAN Variants

- ❑ **Vanilla GAN:** Original formulation
- ❑ **DCGAN:** Deep Convolutional GAN Architectural guidelines for stable training
- ❑ **Conditional GAN:** Conditioning on class labels or attributes
- ❑ **CycleGAN:** Unpaired image-to-image translation
- ❑ **StyleGAN:** Style-based generator for high-quality synthesis
- ❑ **Progressive GAN:** Growing networks progressively for stability
- ❑ **BigGAN:** Large-scale GAN for high-resolution images

GAN

CGAN[39] BS-GAN[48]
 DCGAN[40] Cycle-GAN[49]
 CoGAN[41] AnoGAN[50]
 InfoGAN[42] BEGAN[51]
 SGAN[43] BiGAN[52]
 CC-GAN[44] DualGAN[53]
 AdaGAN[45] StackGAN[54]
 WGAN[46] LSGAN[55]
 AC-GAN[47] ...



G : Generator, **D**: Discriminator, **Q**: Q-network to learn disentangled representation,
C: Classifier Network, X_r : Original Image, X_g : Generated Image

Task

Task: Presenting GAN Variants

Prepare a 10-minute presentation

- **Explain the main types of Generative Adversarial Network (GAN) variants:** including DCGAN, WGAN, CycleGAN, StyleGAN, and Conditional GANs.
- **Highlight practical applications and differences in architecture or training techniques** for each variant, using relevant visuals or examples where possible.

A series of concentric hexagons drawn with thin, light blue lines, creating a tunnel-like effect that draws the eye towards the center. The hexagons are slightly offset from each other, giving a 3D perspective.

06

Transfer learning and pre-trained models

What is transfer learning?

- ❑ **Transfer learning** allows us to take the patterns (also called weights) another model has learned from another problem and use them for our own problem.
- ❑ **For example**, we can take the patterns a computer vision (or other tasks) model has learned from datasets such as **ImageNet** (millions of images of different objects) and use them to power our project.

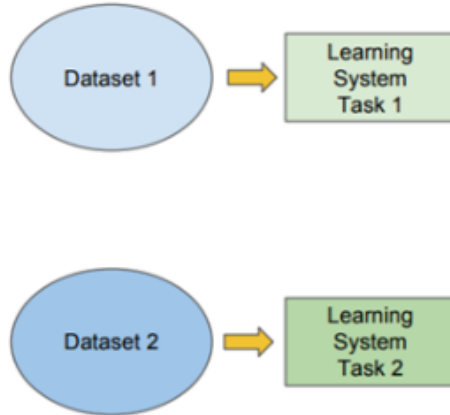
Why use transfer learning?

- ❑ There are two main benefits to using transfer learning:
- ❑ Can leverage an existing model (usually a neural network architecture) proven to work on problems similar to our own.
- ❑ Can leverage a working model which has already learned patterns on similar data to our own. This often results in achieving great results with less custom data.

Traditional ML & Transfer Learning

Traditional ML

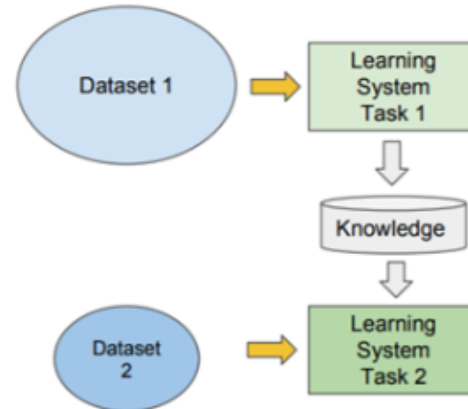
- Isolated, single task learning:
 - Knowledge is not retained or accumulated. Learning is performed w.o. considering past learned knowledge in other tasks



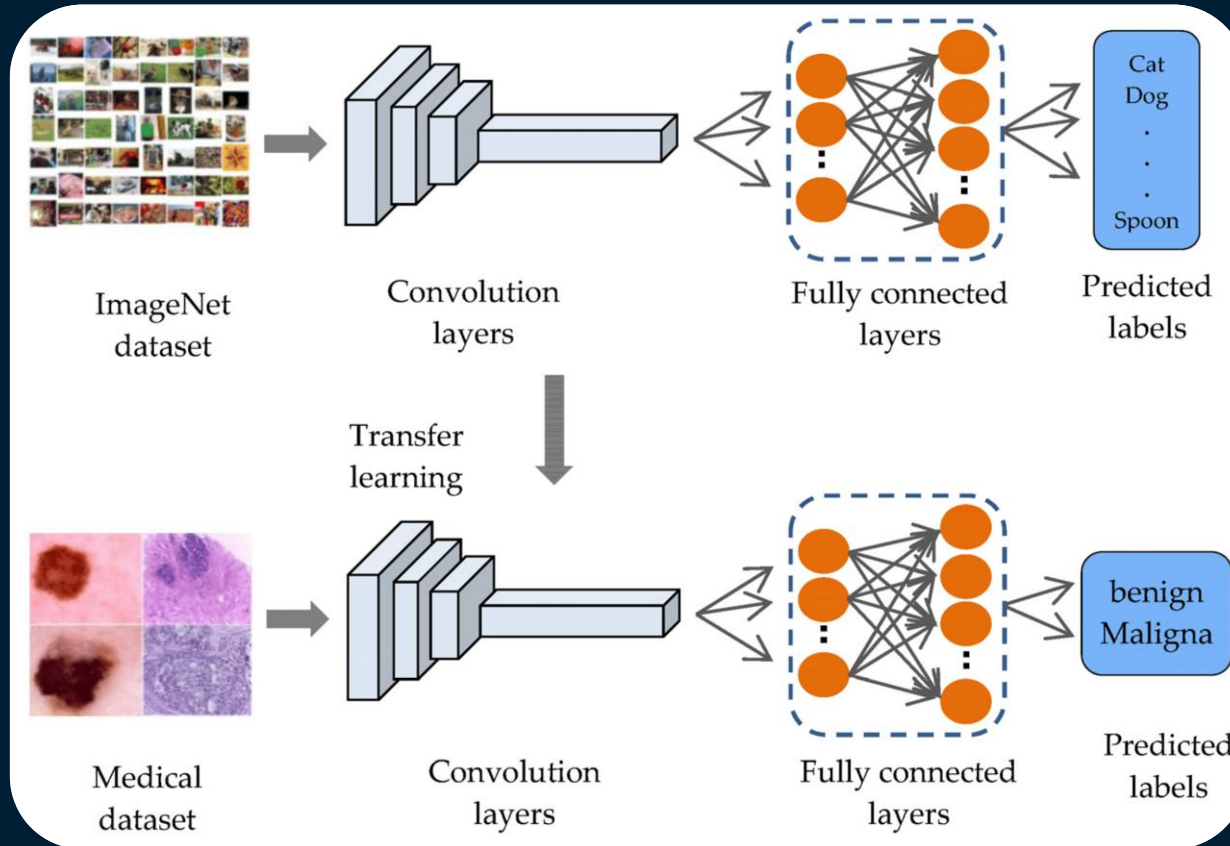
vs

Transfer Learning

- Learning of a new tasks relies on the previous learned tasks:
 - Learning process can be faster, more accurate and/or need less training data

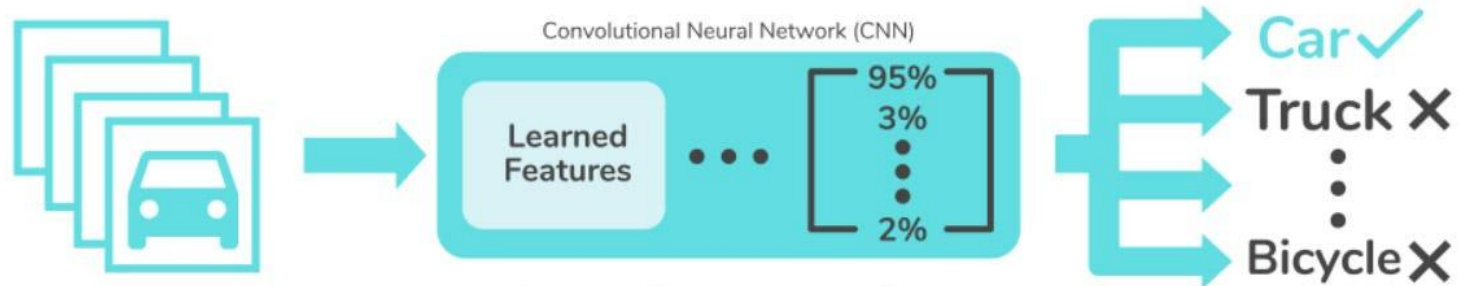


ImageNet for computer vision tasks



Training from scratch & Transfer Learning

Training from Scratch



Transfer Learning



Free Time Learn

What Are Pre-trained Models?

- A pre-trained model is a deep neural network model that has been previously trained on a large dataset and saved for reuse. Rather than training from scratch, you start from a model that has already learned rich, general-purpose feature representations.

Pre-trained Models In Computer Vision

Model	Key Features	Use Cases
ResNet	Residual connections, deep (50, 101, 152 layers)	Classification, detection
VGG	Simple architecture, uniform conv layers	Transfer learning, medical imaging
Inception	Multi-scale convolutions, efficient	Edge devices, general purpose
EfficientNet	Compound scaling of depth, width, resolution	Mobile & high-perf inference

EfficientNet

Compound scaling of depth, width, resolution

Mobile & high-perf inference

Pre-trained Models In NLP



Model	Type	Purpose	Pre-training Objective
BERT	Encoder	Text classification, QA, NER	Masked Language Modeling
GPT (2/3/4)	Decoder	Text generation, chatbots	Causal Language Modeling
RoBERTa	BERT variant	Robust, dynamic masking	MLM (optimized)
DistilBERT	Lightweight	Faster, smaller	Knowledge distillation
T5	Encoder-Decoder	Text2text tasks (QA, summarization)	Text-to-text transfer

12

Encoder-Decoder

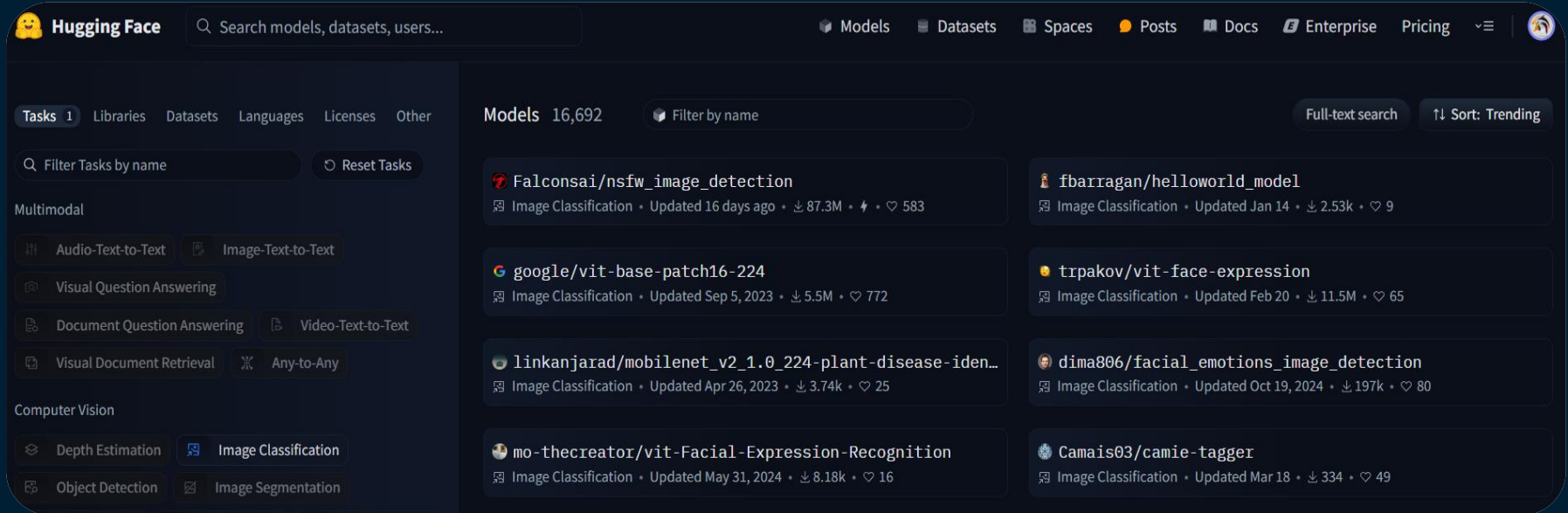
Text2Text tasks (QA, summarization)

Text-to-text transfer

Where to find pretrained models

Location	What's there?	Link(s)
PyTorch domain libraries	Each of the PyTorch domain libraries (<code>torchvision</code> , <code>torchtext</code>) come with pretrained models of some form. The models there work right within PyTorch.	<code>torchvision.models</code> , <code>torchtext.models</code> , <code>torchaudio.models</code> , <code>torchrec.models</code>
HuggingFace Hub	A series of pretrained models on many different domains (vision, text, audio and more) from organizations around the world. There's plenty of different datasets too.	https://huggingface.co/models , https://huggingface.co/datasets
<code>timm</code> (PyTorch Image Models) library	Almost all of the latest and greatest computer vision models in PyTorch code as well as plenty of other helpful computer vision features.	https://github.com/rwightman/pytorch-image-models
Paperswithcode	A collection of the latest state-of-the-art machine learning papers with code implementations attached. You can also find benchmarks here of model performance on different tasks.	https://paperswithcode.com/

My favorite source: Hugging Face



The screenshot displays the Hugging Face homepage. At the top, there's a navigation bar with the Hugging Face logo, a search bar, and links to Models, Datasets, Spaces, Posts, Docs, Enterprise, and Pricing. Below this, a sidebar on the left offers filters for Tasks (Multimodal, Audio-Text-to-Text, Visual Question Answering, Document Question Answering, Video-Text-to-Text, Visual Document Retrieval, Any-to-Any) and Computer Vision (Depth Estimation, Image Classification, Object Detection, Image Segmentation). The main content area shows a grid of model cards. Each card includes the model name, a brief description, update date, size, and popularity metrics (likes and downloads).

Model Name	Description	Updated	Size	Likes	Downloads
Falconsai/nsfw_image_detection	Image Classification	16 days ago	87.3M	583	
fbarragan/helloworld_model	Image Classification	Jan 14	2.53k	9	
google/vit-base-patch16-224	Image Classification	Sep 5, 2023	5.5M	772	
trpakov/vit-face-expression	Image Classification	Feb 20	11.5M	65	
linkanjarad/mobilenet_v2_1.0_224-plant-disease-iden...	Image Classification	Apr 26, 2023	3.74k	25	
dima806/facial_emotions_image_detection	Image Classification	Oct 19, 2024	197k	80	
mo-thecreator/vit-Facial-Expression-Recognition	Image Classification	May 31, 2024	8.18k	16	
Camais03/camie-tagger	Image Classification	Mar 18	334	49	

Model Size and Parameters in Pre-trained Models

Weight	Acc@1	Acc@5	Params	GFLOPS	Recip
AlexNet_Weights.IMAGENET1K_V1	56.522	79.066	61.1M	0.71	link
ConvNeXt_Base_Weights.IMAGENET1K_V1	84.062	96.87	88.6M	15.36	link
ConvNeXt_Large_Weights.IMAGENET1K_V1	84.414	96.976	197.8M	34.36	link
ConvNeXt_Small_Weights.IMAGENET1K_V1	83.616	96.65	50.2M	8.68	link
ConvNeXt_Tiny_Weights.IMAGENET1K_V1	82.52	96.146	28.6M	4.46	link
DenseNet121_Weights.IMAGENET1K_V1	74.434	91.972	8.0M	2.83	link
DenseNet161_Weights.IMAGENET1K_V1	77.138	93.56	28.7M	7.73	link
DenseNet169_Weights.IMAGENET1K_V1	75.6	92.806	14.1M	3.36	link
DenseNet201_Weights.IMAGENET1K_V1	76.896	93.37	20.0M	4.29	link
EfficientNet_B0_Weights.IMAGENET1K_V1	77.692	93.532	5.3M	0.39	link
EfficientNet_B1_Weights.IMAGENET1K_V1	78.642	94.186	7.8M	0.69	link
EfficientNet_B1_Weights.IMAGENET1K_V2	79.838	94.934	7.8M	0.69	link
EfficientNet_B2_Weights.IMAGENET1K_V1	80.608	95.31	9.1M	1.09	link
EfficientNet_B3_Weights.IMAGENET1K_V1	82.008	96.054	12.2M	1.83	link
EfficientNet_B4_Weights.IMAGENET1K_V1	83.384	96.594	19.3M	4.39	link
EfficientNet_B5_Weights.IMAGENET1K_V1	83.444	96.628	30.4M	10.27	link
EfficientNet_B6_Weights.IMAGENET1K_V1	84.008	96.916	43.0M	19.07	link
EfficientNet_B7_Weights.IMAGENET1K_V1	84.122	96.908	66.3M	37.75	link
EfficientNet_V2_L_Weights.IMAGENET1K_V1	85.808	97.788	118.5M	56.08	link
EfficientNet_V2_M_Weights.IMAGENET1K_V1	85.112	97.156	54.1M	24.58	link
EfficientNet_V2_S_Weights.IMAGENET1K_V1	84.228	96.878	21.5M	8.37	link
GoogLeNet_Weights.IMAGENET1K_V1	69.778	89.53	6.6M	1.5	link
Inception_V3_Weights.IMAGENET1K_V1	77.294	93.45	27.2M	5.71	link
MNASNet0_5_Weights.IMAGENET1K_V1	67.734	87.49	2.2M	0.1	link
MNASNet0_75_Weights.IMAGENET1K_V1	71.18	90.496	3.2M	0.21	link
MNASNet1_0_Weights.IMAGENET1K_V1	73.456	91.51	4.4M	0.31	link
MNASNet1_3_Weights.IMAGENET1K_V1	76.506	93.522	6.3M	0.53	link
MaxViT_T_Weights.IMAGENET1K_V1	83.7	96.722	30.9M	5.56	link
MobileNet_V2_Weights.IMAGENET1K_V1	71.878	90.286	3.5M	0.3	link

FileNet_V2_X1_0_Weights.IMAGENET1K_V1	69.362	88.316	2.3M	0.14	link
ShuffleNet_V2_X1_5_Weights.IMAGENET1K_V1	72.996	91.086	3.5M	0.3	link
ShuffleNet_V2_X2_0_Weights.IMAGENET1K_V1	76.23	93.006	7.4M	0.58	link
SqueezeNet1_0_Weights.IMAGENET1K_V1	58.092	80.42	1.2M	0.82	link
SqueezeNet1_1_Weights.IMAGENET1K_V1	58.178	80.624	1.2M	0.35	link
Swin_B_Weights.IMAGENET1K_V1	83.582	96.64	87.8M	15.43	link
Swin_S_Weights.IMAGENET1K_V1	83.196	96.36	49.6M	8.74	link
Swin_T_Weights.IMAGENET1K_V1	81.474	95.776	28.3M	4.49	link
Swin_V2_B_Weights.IMAGENET1K_V1	84.112	96.864	87.9M	20.32	link
Swin_V2_S_Weights.IMAGENET1K_V1	83.712	96.816	49.7M	11.55	link
Swin_V2_T_Weights.IMAGENET1K_V1	82.072	96.132	28.4M	5.94	link
VGG11_BN_Weights.IMAGENET1K_V1	70.37	89.81	132.9M	7.61	link
VGG11_Weights.IMAGENET1K_V1	69.02	88.628	132.9M	7.61	link
VGG13_BN_Weights.IMAGENET1K_V1	71.586	90.374	133.1M	11.31	link
VGG13_Weights.IMAGENET1K_V1	69.928	89.246	133.0M	11.31	link
VGG16_BN_Weights.IMAGENET1K_V1	73.36	91.516	138.4M	15.47	link
VGG16_Weights.IMAGENET1K_V1	71.592	90.382	138.4M	15.47	link
VGG16_Weights.IMAGENET1K_FEATURES	nan	nan	138.4M	15.47	link
VGG19_BN_Weights.IMAGENET1K_V1	74.218	91.842	143.7M	19.63	link
VGG19_Weights.IMAGENET1K_V1	72.376	90.876	143.7M	19.63	link
ViT_B_16_Weights.IMAGENET1K_V1	81.072	95.318	86.6M	17.56	link
ViT_B_16_Weights.IMAGENET1K_SWAG_E2E_V1	85.304	97.65	86.9M	55.48	link
ViT_B_16_Weights.IMAGENET1K_SWAG_LINEAR_V1	81.886	96.18	86.6M	17.56	link
ViT_B_32_Weights.IMAGENET1K_V1	75.912	92.466	88.2M	4.41	link
ViT_H_14_Weights.IMAGENET1K_SWAG_E2E_V1	88.552	98.694	633.5M	1016.72	link
ViT_H_14_Weights.IMAGENET1K_SWAG_LINEAR_V1	85.708	97.73	632.0M	167.29	link
ViT_L_16_Weights.IMAGENET1K_V1	79.662	94.638	304.3M	61.55	link
ViT_L_16_Weights.IMAGENET1K_SWAG_E2E_V1	88.064	98.512	305.2M	361.99	link
ViT_L_16_Weights.IMAGENET1K_SWAG_LINEAR_V1	85.146	97.422	304.3M	61.55	link
ViT_L_32_Weights.IMAGENET1K_V1	76.972	93.07	306.5M	15.38	link
Wide_ResNet101_2_Weights.IMAGENET1K_V1	78.848	94.284	126.9M	22.75	link

How to calculate GPU requirement?

- ❑ In deep learning, GPU memory (VRAM) is crucial for storing the model parameters and intermediate calculations during training and inference. For fine-tuning models, especially large-scale ones like image-to-text models or LLMs, estimating the memory requirements is critical to ensure the selected GPU can handle the task without running into memory constraints.

$$M = (32/Q) \times (P \times 4B) \times 1.2 \quad (4)$$

Where:

- M is the GPU memory required in GB.
- P is the number of model parameters in billions.
- Q is the precision in bits (commonly 16, 8, or 4).
- 4B represents 4 bytes (since each parameter occupies 4 bytes when stored in 32-bit precision).

Task

 **Task: Presenting Diffusion Model Variants**

 **Prepare a 10-minute presentation**

 **Objective: Explain the main types and advancements of Diffusion Models in generative modeling.**

Thank You

Do you have any questions?

pejman.ebrahimi@uni.li

